Especificação de Projeto: Plataforma TicketRush em Alta Disponibilidade

Infraestruturas de Computação na Nuvem · Projeto integrador em grupo (Aulas 01–06)

Referência	ICN-PI-2026-01, versão 1.0
Modalidade	Exercício em grupo
Conceitos mobilizados	Datacenter e redundância (A01–A02), virtualização (A03), contentores (A04), orquestração (A05), rede centralizada e microssegmentação (A06)
Avaliação	Participação (secção 9)

1. Objeto

O presente documento especifica os requisitos a que deve obedecer a infraestrutura da plataforma **TicketRush**, serviço de venda de bilhetes e controlo de acessos da Queima das Fitas, e define as etapas do exercício que cada grupo resolve. A aplicação está desenvolvida e fornecida (pasta `app/`). **O objeto do trabalho é a infraestrutura que a hospeda**: redundância, réplicas, persistência e isolamento.

2. Contexto operacional

1. O serviço é de **missão crítica durante a janela de vendas**: cada minuto de indisponibilidade é venda perdida e reclamações públicas.
2. A abertura das vendas gera um **pico de carga** estimado em 50 vezes a carga média, durante cerca de 15 minutos.
3. A lotação de cada evento é limitada: **é inadmissível vender mais bilhetes do que a lotação**.
4. As encomendas confirmadas são **dados com valor legal e financeiro**: não podem ser perdidas.
5. O sistema processa dados pessoais: o acesso à camada de dados deve ser **restrito e auditável**.

3. Sistema existente (ponto de partida)

A aplicação funciona hoje num único servidor, sem qualquer redundância (Figura 1).

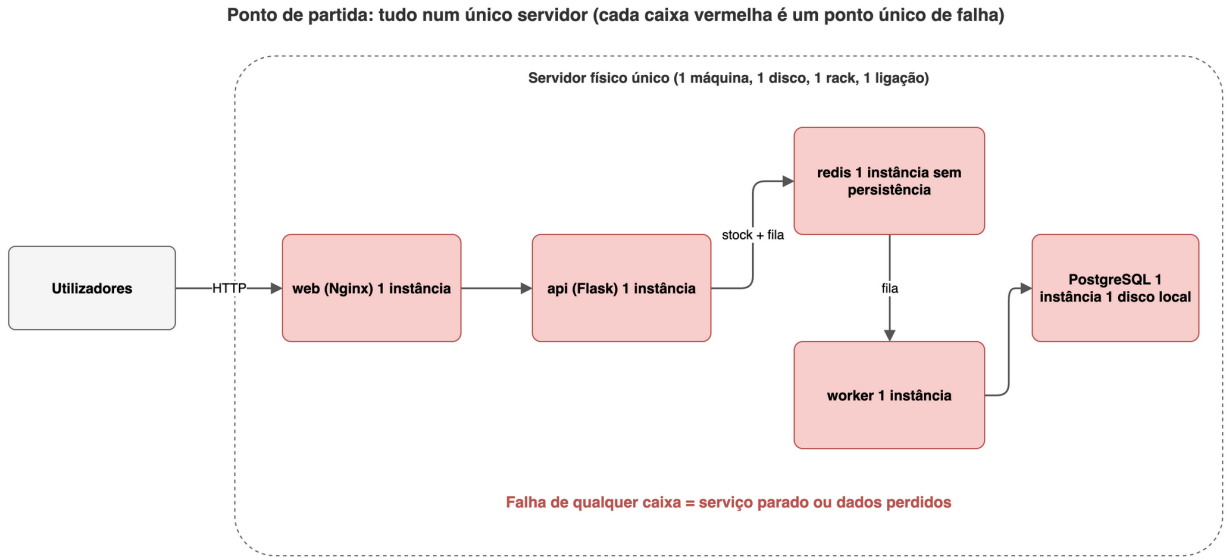


Figura 1: Ponto de partida: cada componente é um ponto único de falha.

Componente	Instâncias	Persistência	Ponto único de falha
`web` (Nginx)	1	n/a	Sim
`api` (Flask)	1	sem estado	Sim
`redis` (stock e fila)	1	**nenhuma**	Sim; perda do stock e dos pedidos em fila
`worker`	1	sem estado	Sim; a fila deixa de ser consumida
`postgres`	1	disco local do servidor	Sim; perda das encomendas
Servidor, disco, rack, ligação de rede	1	n/a	Sim

O percurso de uma reserva, e os pontos onde cada dado é guardado, está na Figura 2.

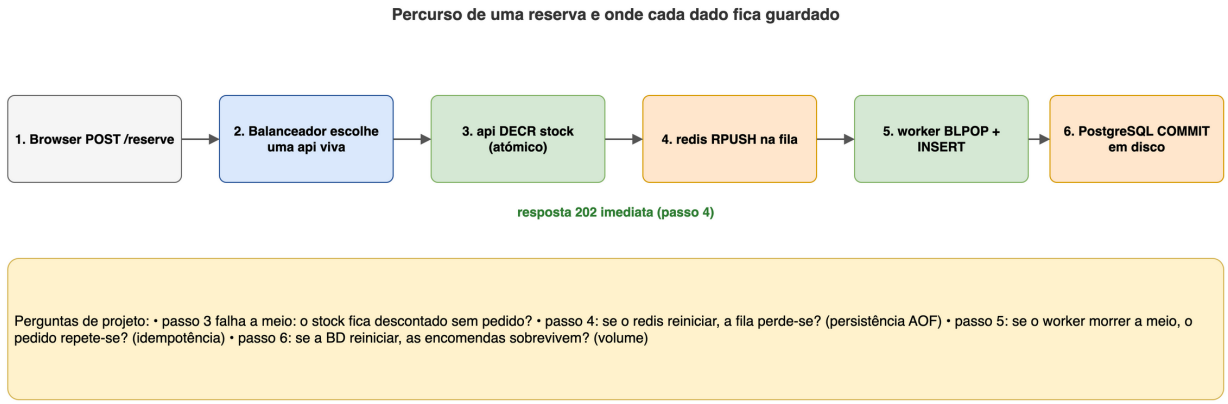


Figura 2: Percurso de uma reserva.

4. Requisitos funcionais

ID	Requisito
RF-01	Listar eventos e o stock disponível
RF-02	Reservar um bilhete (resposta imediata; confirmação assíncrona)
RF-03	Consultar o estado de uma encomenda
RF-04	O stock nunca pode ficar negativo, mesmo sob concorrência

5. Requisitos não funcionais

ID	Requisito	Critério de verificação
RNF-01	**Disponibilidade** de 99,95 % durante a janela de vendas	A falha de qualquer componente individual não interrompe o serviço
RNF-02	**Redundância N+1** em todos os níveis: instâncias de aplicação, balanceamento, dados, servidores e sites	Nenhum ponto único de falha na arquitetura-alvo (Etapa 2)
RNF-03	**Réplicas de aplicação**: mínimo de 3 réplicas da `api` e 2 do `worker`, repartidas por domínios de falha distintos	`docker compose ps` mostra as réplicas; a carga é repartida
RNF-04	**Persistência**: encomendas e fila sobrevivem ao reinício de qualquer contentor	Teste de reinício (T3, T4, definidos na Etapa 4)
RNF-05	**RPO ≤ 5 min e RTO ≤ 15 min** em perda total de um site	Estratégia de replicação e de *backup* justificada na Etapa 2
RNF-06	**Isolamento de rede** em zonas, com política *default deny*	Matriz de fluxos (Etapa 3) e teste T5
RNF-07	**Isolamento de recursos**: limites de CPU e memória por serviço	Campo `deploy.resources` no Compose
RNF-08	**Gestão de segredos**: nenhuma credencial no código nem no repositório	Ficheiro `.env` ignorado pelo controlo de versões
RNF-09	**Observabilidade mínima**: *healthcheck* em todos os serviços de dados	`docker compose ps` mostra `healthy`

6. Etapas do exercício

O exercício é resolvido **em grupo**; não há entrega.

Etapa 1: Análise de riscos

Para cada linha da tabela da secção 3: o que acontece quando falha, quão provável é, que impacto tem e como se mitiga. Identificar os **domínios de falha** (contentor, máquina, rack, site) e que componentes partilham cada um.

Etapa 2: Arquitetura-alvo

Completar em grupo o modelo da Figura 3, substituindo cada «?» por uma decisão **justificada**:

- n.º de balanceadores e como se deteta e reage a uma falha (VIP, verificações de saúde);
- n.º de réplicas da `api` e do `worker`, e porquê;
- replicação do Redis e do PostgreSQL (primário/réplica; síncrona ou assíncrona; efeito no RPO);
- onde residem os volumes persistentes e como se replicam entre máquinas e sites;
- modo do segundo site (ativo-ativo ou ativo-*standby*) e mecanismo de *failover*;
- política de *backup*: frequência, retenção, local das cópias, como se testa o restauro.

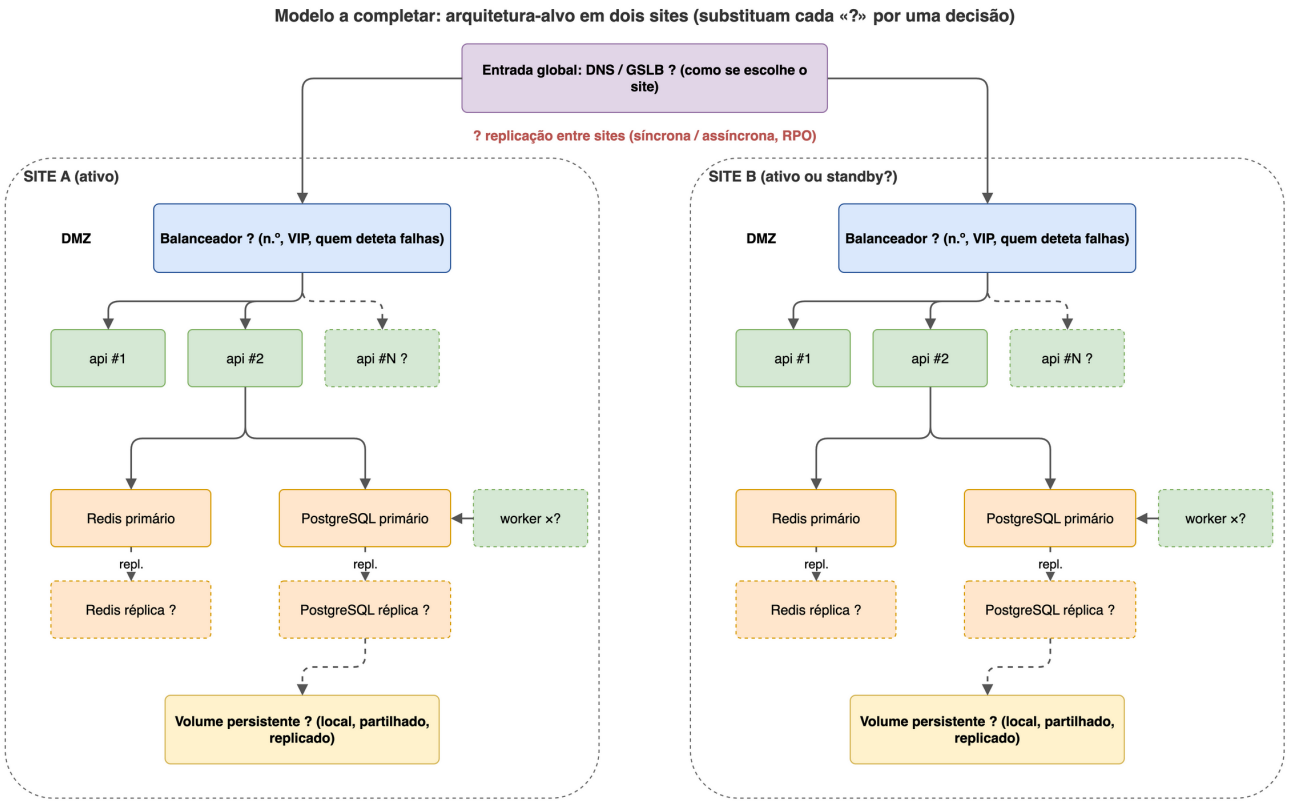


Figura 3: Modelo de arquitetura-alvo a completar.

Etapa 3: Segmentação e isolamento de rede

Preencher a matriz de fluxos (origem, destino, porto, permitido/negado) para as zonas da Figura 4, com política *default deny*. Discutir em que tecnologia se concretiza cada regra (redes Docker `internal`, regras de *firewall*, `NetworkPolicy` em Kubernetes) e porque é que a zona de dados não tem saída para a Internet.

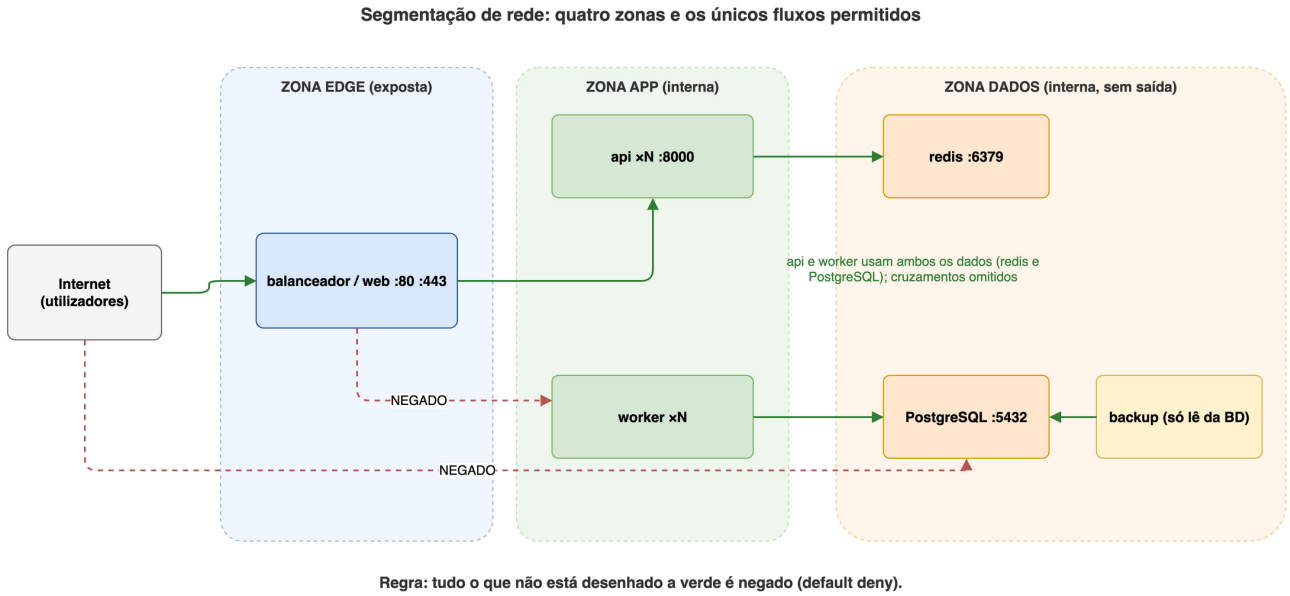


Figura 4: Zonas de rede e fluxos permitidos.

Etapa 4: Preparação dos testes de falha

Esta etapa fecha o planeamento: **antes de escrever uma linha de Compose**, o grupo decide como vai provar que a arquitetura aguenta as avarias. Para cada ponto único de falha ou requisito das Etapas 1 a 3, define-se um teste com:

- **cenário de avaria** (o que se destrói, pára ou bloqueia, e com que comando);
- **carga ou estado de partida** (por exemplo, a decorrer `./carga.sh 100`);
- **resultado esperado**, **mensurável** (nº de pedidos aceites, dados presentes, ligação recusada);
- **como se observa** (comando ou registo que mostra o resultado);
- **requisito que o teste prova** (RNF-xx).

Ponto de partida sugerido; o grupo pode adotar, alterar ou acrescentar testes:

ID	Cenário de avaria	Resultado esperado
T1	`docker kill` de uma réplica da `api` durante `./carga.sh 100`	Sem pedidos falhados de forma permanente; a réplica volta
T2	Paragem do `worker` com a fila a encher; depois arranque	Todos os pedidos acabam por ser gravados, sem duplicados
T3	`docker compose restart db`	Encomendas anteriores intactas
T4	`docker compose restart redis`	Stock e fila preservados (AOF)
T5	`docker compose exec web nc -zv db 5432`	**Falha** (isolamento)
T6	`./carga.sh 400 3` (evento com 50 lugares)	Exatamente 50 aceites; os restantes `esgotado`

Cada teste planeado tem de corresponder a pelo menos um risco da Etapa 1 e a um requisito da secção 5. Um requisito sem teste associado é uma lacuna do planeamento.

Etapa 5: Implementação

Escrever em grupo, no VS Code, o `docker-compose.yml` que levanta o sistema a partir de `app/` e realiza os requisitos RNF-03, RNF-04 e RNF-06 a RNF-09:

- serviços `web`, `api`, `worker`, `redis`, `db` e um serviço `backup`;
- redes `edge`, `app` e `data`, estas duas últimas `internal`;
- volumes nomeados para a base de dados, para o Redis (com AOF ativo) e para as cópias de segurança (separado do volume dos dados);
- `healthcheck` e `depends_on` com `condition: service_healthy`;
- `restart: unless-stopped`, réplicas e limites de recursos;
- credenciais lidas de `.env`.

Etapa 6: Execução dos testes de falha

Executar o plano da Etapa 4 sobre o sistema implementado: provocar cada avaria, observar e comparar com o resultado esperado. Cada resultado que diverge é discutido: o plano estava errado, ou falhou a arquitetura ou o Compose?

Etapa 7: Balanço

Discussão final: que pontos únicos de falha ainda subsistem (o site, o Redis, o balanceador) e o que seria preciso para os eliminar?

7. Metodologia

O exercício decorre em cinco fases (planeamento, incluindo os testes de falha, e depois implementação), todas realizadas em grupo (Figura 5): o desenho em quadro partilhado (Excalidraw) e a implementação em edição simultânea (VS Code Live Share).

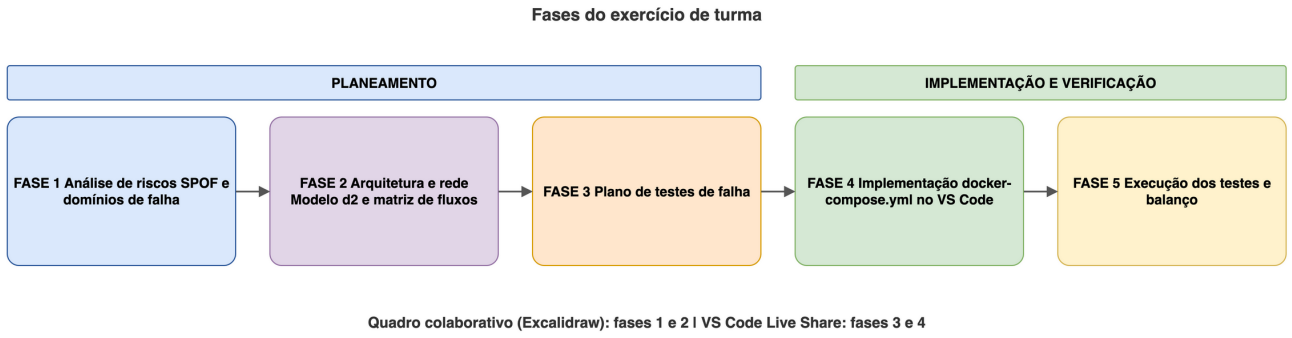


Figura 5: Faseamento.

8. Restrições

1. A aplicação em `app/` não pode ser alterada: o exercício é só de infraestrutura.
2. Todo o ambiente deve arrancar com `docker compose up -d --build` numa máquina com Docker.
3. Não são admitidas credenciais, chaves ou palavras-passe em ficheiros versionados.
4. Cada decisão de arquitetura tem de ter uma justificação (uma frase basta).

9. Avaliação

A avaliação baseia-se **unicamente na participação** de cada aluno ao longo do exercício: propostas no quadro, votações, intervenções na discussão e contributos para o ficheiro comum. Não há entregáveis nem relatório.